

ST. PETERSBURG STATE UNIVERSITY
Faculty of Mathematics and Mechanics

L/R No. 4: Parallel.
Made by Lichkovakh Daniil Andreevich 24 B 81-mm.

Teachers:
Vyacheslav Grokhovsky

Figure 1 shows a clearly improved performance.

For the first image, the clockwise rotation was accelerated by 2 ms, counterclockwise by 3 ms. The application of the blurr was accelerated by 40 ms, that is, it is now done 3 times faster.

For the second image, the clockwise rotation was accelerated by 17 ms, counterclockwise by 16 ms. The application of the blurr was accelerated by 139 ms.

```
1 thread:
for rotateC took 10 ms
for rotateCC took 5 ms
for blur took 60 ms
-----
2nd image:
1 thread:
for rotateC took 25 ms
for rotateCC took 31 ms
for blur took 240 ms
-----
-----
1st image:
4 thread:
for rotateC took 8 ms
for rotateCC took 2 ms
for blur took 20 ms
-----
-----
2nd image:
4 thread:
for rotateC took 8 ms
for rotateCC took 15 ms
for blur took 71 ms
daniillichkovaha@daniillichkovaha:~/Documents/LabWork1$
```

Fig. 1 - the result of the acceleration of the program.

Explanation of the drawing:

"rotateC" is a rotate clockwise

"rotateCC" is a rotate counter clockwise

blur is Gaussian Blur

It can be stated that parallelization has significantly reduced the execution time of the program. It is worth noting that the tests were carried out thanks to the testParal function with the following implementation:

```

#include "BMP.h"
#include "testParal.h"
#include <iostream>
#include <string>
#include <chrono>

void testParal(BMP& img, const std::string& task) {
    auto start = std::chrono::high_resolution_clock::now(); // start

    if (task == "rotateC") {
        img.Rotate90Clockwise();
        img.Save("Rotated90Clockwise.bmp");
    } else if (task == "rotateCC") {
        img.Rotate90CounterClockwise();
        img.Save("Rotate90CounterClockwise.bmp");
    } else if (task == "blur") {
        img.GaussianBlur();
        img.Save("GaussianBlur.bmp");
    }

    auto end = std::chrono::high_resolution_clock::now(); // end
    auto dur = std::chrono::duration_cast<std::chrono::milliseconds>(end - start); // duration

    std::cout << "for " << task << " | took " << dur.count() << " ms" << std::endl;
}

```

Fig. 2 - implementation of the testParal function.